

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Universidad Tecnológica Nacional — Modalidad a Distancia

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Materia: Programación 1

Integrantes: Sebastian Molina
Andrés Manríquez

Fecha de entrega: 15 de junio de 2025

Profesores: ANA EUGENIA MUTTI

Índice

1. Introducción	3
2. Marco Teórico	3
2.1 Listas	3
2.2 Diccionarios	3
2.3 Funciones	4
2.4 Estructuras condicionales y repetitivas	4
2.5 Ordenamientos	4
2.6 Estadísticas básicas	5
2.7 Archivos CSV	5
3. Decisiones Técnicas y Arquitectura	5
3.1 Estructura del proyecto	5
3.2 Diagrama de flujo	6
4. Funcionalidades del sistema	6
5. Validaciones implementadas	7
6. Dificultades y Conclusiones	7
7. Bibliografía / Webgrafía	8

1. Introducción

Este trabajo práctico integrador consiste en el desarrollo de una aplicación en Python para gestionar información sobre países del mundo. El sistema permite cargar datos desde un archivo CSV, y ofrece funcionalidades de búsqueda, filtrado, ordenamiento y estadísticas a través de un menú interactivo en consola.

El objetivo principal es aplicar los conceptos aprendidos en Programación 1: listas, diccionarios, funciones, estructuras condicionales y repetitivas, manejo de archivos y validación de datos.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos. Se definen con corchetes y pueden contener elementos de cualquier tipo, incluyendo otros objetos como diccionarios. En este proyecto, la lista principal almacena todos los países del sistema:

```
países = [] # lista vacía que luego contiene diccionarios
```

Las listas permiten recorrer sus elementos con bucles for, agregar con `.append()`, copiar con `[:]` y ordenar con `.sort()`.

2.2 Diccionarios

Un diccionario es una estructura de datos que almacena pares clave-valor. Cada país del sistema se representa como un diccionario con cuatro claves:

```
pais = {'nombre': 'Argentina', 'poblacion': 45376763, 'superficie': 2780400,  
        'continente': 'América'}
```

El acceso a los valores se hace con `pais['nombre']`, lo que hace el código más legible que si se usaran listas simples con índices numéricos.

2.3 Funciones

Las funciones permiten dividir el código en bloques reutilizables, cada uno con una responsabilidad específica. En Python se definen con la palabra clave `def`. Este proyecto sigue el principio de una función = una responsabilidad, lo que hace el código más fácil de leer, mantener y corregir:

```
def buscar_por_nombre(países): # solo se encarga de buscar ...  
def estadisticas(países): # solo se encarga de calcular estadísticas ...
```

2.4 Estructuras condicionales y repetitivas

Las estructuras condicionales (`if / elif / else`) permiten tomar decisiones en el código según el valor de una expresión. Las estructuras repetitivas (`for, while`) permiten ejecutar un bloque de código varias veces. En este sistema se combinan ambas:

```
while True: # bucle principal del menú opcion = input() if opcion == '1': #  
condicional para cada opción listar_paises(paises) elif opcion == '9': break  
# salida del bucle
```

2.5 Ordenamientos

Python ofrece el método `.sort()` para ordenar listas en el lugar, y la función `sorted()` para obtener una nueva lista ordenada. Ambas aceptan el parámetro `key` para indicar por qué criterio ordenar, y `reverse=True` para orden descendente:

```
# Ordenar por población de mayor a menor copia.sort(key=lambda x:  
x['poblacion'], reverse=True)
```

En este proyecto se trabaja siempre sobre una copia de la lista original para no modificar los datos mientras se muestran resultados.

2.6 Estadísticas básicas

Las estadísticas se calculan recorriendo la lista con un bucle `for` y actualizando variables acumuladoras. Se calculan: máximo, mínimo, promedio y conteo por categoría. El promedio se obtiene dividiendo la suma total por la cantidad de elementos:

```
suma_pob = 0 for p in paises: suma_pob += p['poblacion'] promedio = suma_pob  
// len(paises)
```

2.7 Archivos CSV

CSV (Comma Separated Values) es un formato de texto simple para almacenar datos tabulares. Python incluye el módulo `csv` en su biblioteca estándar, que facilita la lectura y escritura de estos archivos. Se usa `csv.DictReader` para leer cada fila como un diccionario, y `csv.DictWriter` para escribir:

```
import csv with open('paises.csv', 'r', encoding='utf-8') as archivo: lector  
= csv.DictReader(archivo, delimiter=';') for fila in lector:  
paises.append(fila)
```

3. Decisiones Técnicas y Arquitectura

3.1 Estructura del proyecto

El proyecto se dividió en dos archivos para separar responsabilidades:

Archivo	Responsabilidad
main.py	Menú principal, flujo del programa, interacción con el usuario
funciones.py	Todas las funciones de lógica: agregar, buscar, filtrar, ordenar, estadísticas
países.csv	Dataset base con 12 países del mundo

3.2 Diagrama de flujo del programa

El flujo principal del sistema sigue este esquema:

Paso	Acción	Resultado
1	Inicio del programa	Carga automática del CSV
2	Muestra el menú	Usuario elige opción 1-9
3	Opción 1	Lista todos los países
3	Opción 2	Solicita datos → valida → agrega
3	Opción 3	Busca país → actualiza población/superficie
3	Opción 4	Busca por nombre parcial o exacto
3	Opción 5	Submenú: filtrar por continente/población/superficie
3	Opción 6	Submenú: ordenar por nombre/población/superficie
3	Opción 7	Calcula y muestra estadísticas
3	Opción 8	Guarda cambios en el CSV
4	Opción 9	Pregunta si guardar → finaliza el programa

4. Funcionalidades del sistema

Función	Descripción
cargar_datos()	Lee el CSV y carga los países en memoria. Detecta el separador automáticamente.
guardar_datos()	Escribe la lista de países actualizada en el CSV.
listar_paises()	Muestra todos los países en formato de tabla alineada.
agregar_pais()	Solicita y valida todos los datos. No permite duplicados ni campos vacíos.
actualizar_pais()	Busca un país y permite modificar su población y/o superficie.
buscar_por_nombre()	Búsqueda parcial o exacta por nombre, sin distinción de mayúsculas.
filtrar_por_continente()	Filtra países que pertenecen al continente indicado.
filtrar_por_poblacion()	Filtra países dentro de un rango de población.
filtrar_por_superficie()	Filtra países dentro de un rango de superficie.
ordenar_paises()	Ordena por nombre, población o superficie (asc/desc). Trabaja sobre copia.
estadisticas()	Calcula máx, mín, promedio de población y superficie, y cantidad por continente.
normalizar_texto()	Quita tildes para comparar sin importar acentuación.
pedir_nombre()	Pide el nombre en bucle: no acepta números ni duplicados, aplica .title().
pedir_entero_positivo()	Pide un número en bucle: acepta formato con comas, rechaza letras y negativos.
pedir_continente()	Pide el continente en bucle validando contra la lista de continentes válidos.

5. Validaciones implementadas

Campo	Validación
Nombre del país	No puede estar vacío. No puede contener números. No puede ser duplicado. Se guarda con la primera letra en mayúscula (.title()).
Población	Solo acepta números enteros positivos. Acepta formato con comas (45,000,000). Rechaza letras y valores negativos o cero.
Superficie	Mismas validaciones que población.
Continente	Debe ser uno de los continentes válidos. Acepta con o sin tildes y sin distinción de mayúsculas. Se guarda con la forma oficial.
CSV al cargar	Controla FileNotFoundError, ValueError en datos numéricos y KeyError en columnas faltantes.
Filtros	Controla que el mínimo no sea mayor al máximo. Informa si no hay resultados sin romper el programa.
Búsqueda vacía	Si el usuario no ingresa texto, muestra error y no ejecuta la búsqueda.

6. Dificultades y Conclusiones

Dificultades encontradas

Una de las primeras dificultades fue el manejo del archivo CSV, que en algunos sistemas usa punto y coma como separador en lugar de coma. Para resolverlo se implementó una detección automática del separador al momento de abrir el archivo.

Otra dificultad fue la validación de entradas del usuario. En un principio, si el usuario ingresaba un dato incorrecto el programa mostraba el error y volvía al menú principal, obligando a repetir todo el proceso. Se corrigió usando bucles while True en cada función de entrada, de modo que el programa repite la pregunta hasta recibir un dato válido.

También se detectó que los nombres con tildes en los continentes (África, América) no coincidían al comparar con texto ingresado sin tildes. Se creó la función normalizar_texto() que elimina los acentos antes de comparar, resolviendo el problema sin perder la presentación correcta de los datos.

Conclusiones

Este proyecto nos permitió aplicar de forma integrada todos los conceptos vistos en Programación 1. Entendimos la importancia de la modularización: separar el código en funciones pequeñas con una sola responsabilidad hace que el programa sea mucho más fácil de corregir y ampliar.

También aprendimos que las validaciones son una parte fundamental del desarrollo, no un agregado opcional. Un programa que no valida correctamente las entradas puede fallar de formas inesperadas. El proceso de 'test de rudeza', probando qué pasa cuando se ingresan datos incorrectos, nos ayudó a encontrar y corregir varios errores que no habíamos considerado inicialmente.

La experiencia de trabajar en equipo a distancia también fue un aprendizaje: usar GitHub para compartir y versionar el código resultó muy útil para coordinar los cambios sin pisarnos el trabajo.

7. Bibliografía / Webgrafía

- [1] Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
- [2] Python Software Foundation. (2024). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- [3] Python Software Foundation. (2024). Built-in Types — Lists. <https://docs.python.org/3/library/stdtypes.html#lists>
- [4] Python Software Foundation. (2024). Defining Functions. <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- [5] Real Python. (2024). Python's .sort() vs sorted(). <https://realpython.com/python-sort/>
- [6] Real Python. (2024). Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- [7] UTN TUP a Distancia. (2026). Material de cátedra Programación 1, Cohorte Marzo 2026.
- [8] <https://www.worldometers.info/es/poblacion-mundial/poblacion-por-pais/>